



UNIVERSITÀ
degli STUDI
di CATANIA

ESP32: IoT



davide.patti@dieei.unict.it

References

Most examples will be carried on the Wokwi simulator, but are directly applicable to the physical microcontroller with the Arduino IDE configured for running the ESP32

Simulator: <https://wokwi.com/esp32>

Tutorials: <https://esp32io.com/>

Check the components available at unict:

<https://bit.ly/labunict>

Enabling WiFi con ESP32

- In general, include the library and provide the credentials
- In the example: with the Wokwi simulator, use the following fake credentials to access the a virtual wifi network (replace with actual data when using a physical device).

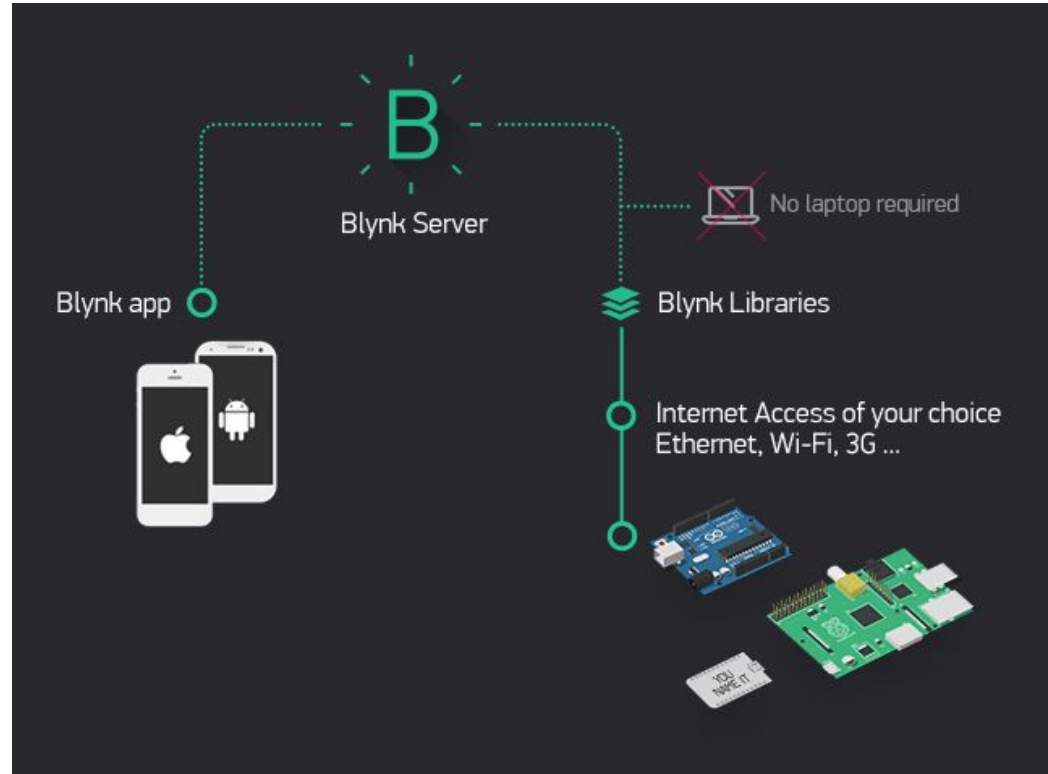
```
#include <WiFi.h>

void setup() {
  Serial.begin(9600);
  Serial.print("Connecting to WiFi");
  WiFi.begin("Wokwi-GUEST", "", 6);
  while (WiFi.status() != WL_CONNECTED) {
    delay(100);
    Serial.print(".");
  }
  Serial.println(" Connected!");
}

void loop() {
  delay(100); // TODO: Build something amazing!
}
```

Blynk

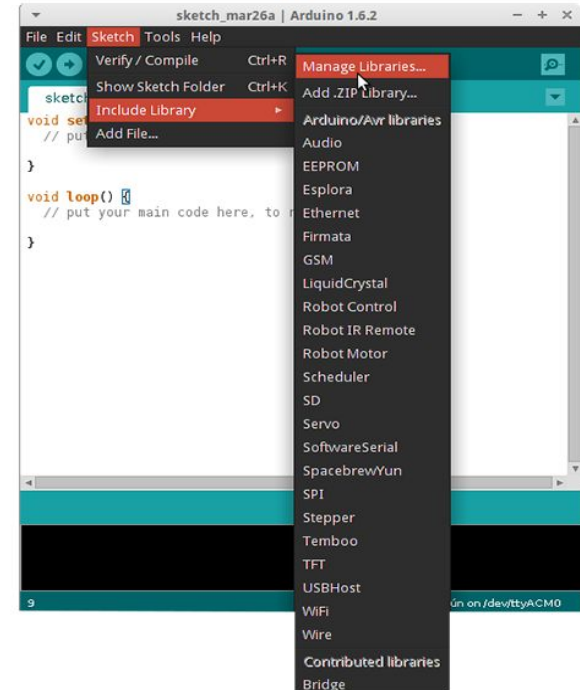
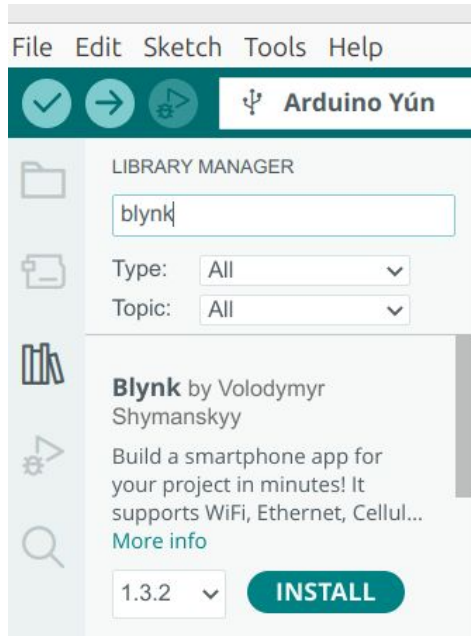
- IoT Platform
- Allows devices to be connected through the Blynk cloud



Blynk Library (Physical Device)

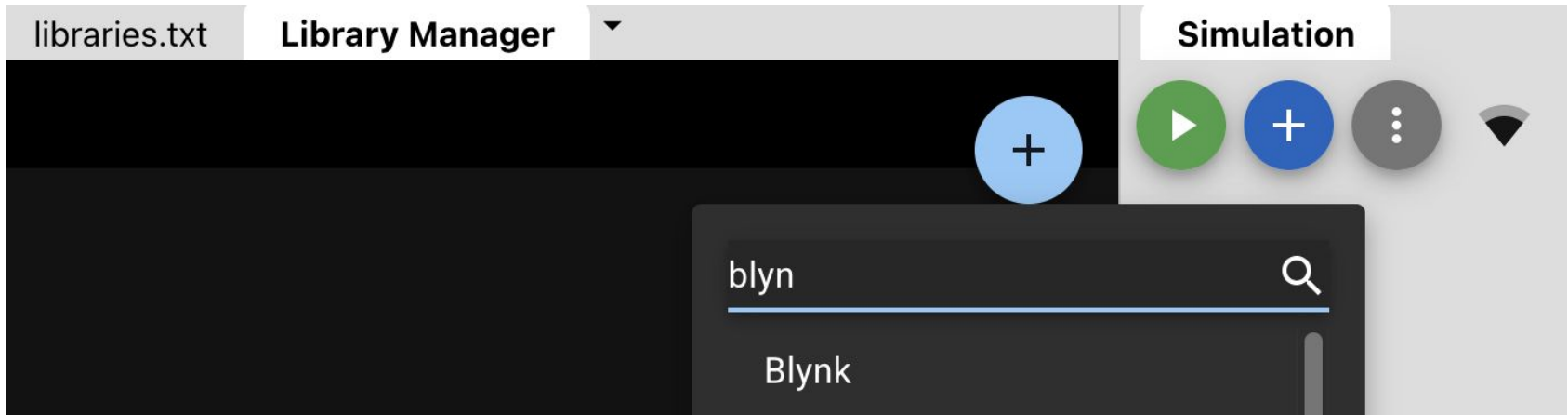
Add Blynk library to Arduino IDE:

<https://docs.blynk.io/en/blynk-library-firmware-api/installation/install-blynk-library-in-arduino-ide>



Blynk Library (Wokwi)

The blynk library must be added to the simulator



Blynk.Console

Accesso

 <https://blynk.io/>

Modalità Operative

 **User Mode**

 **Developer Mode**

Dettagli Developer Mode

 Crea e configura i **Device Templates**

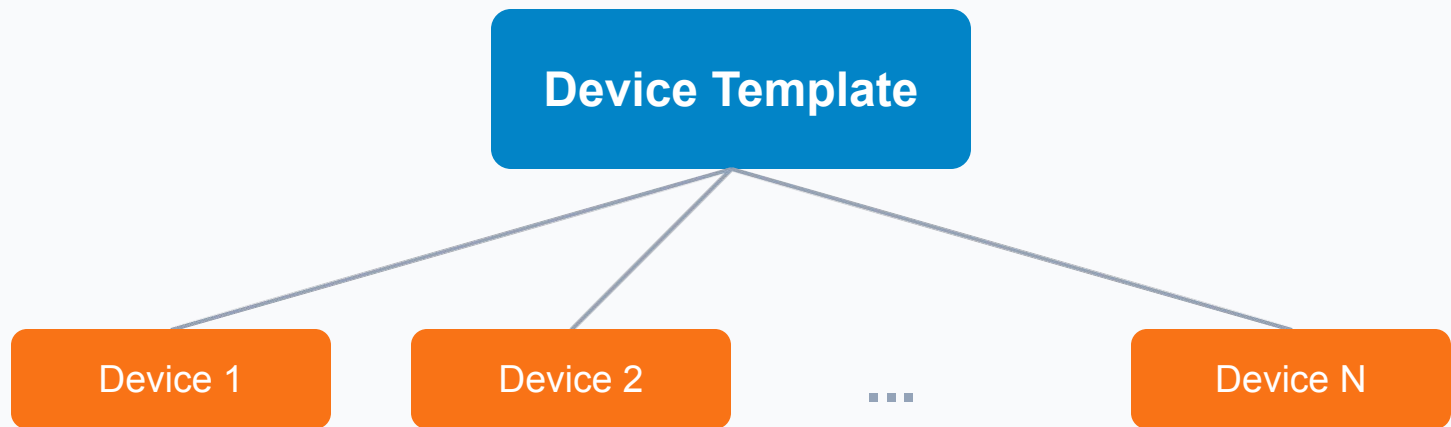
 Crea e configura la **dashboard UI** (web/mobile)

 Aggiungi **nuovi dispositivi** all'account

 Accedi alle funzionalità in base al **piano scelto**

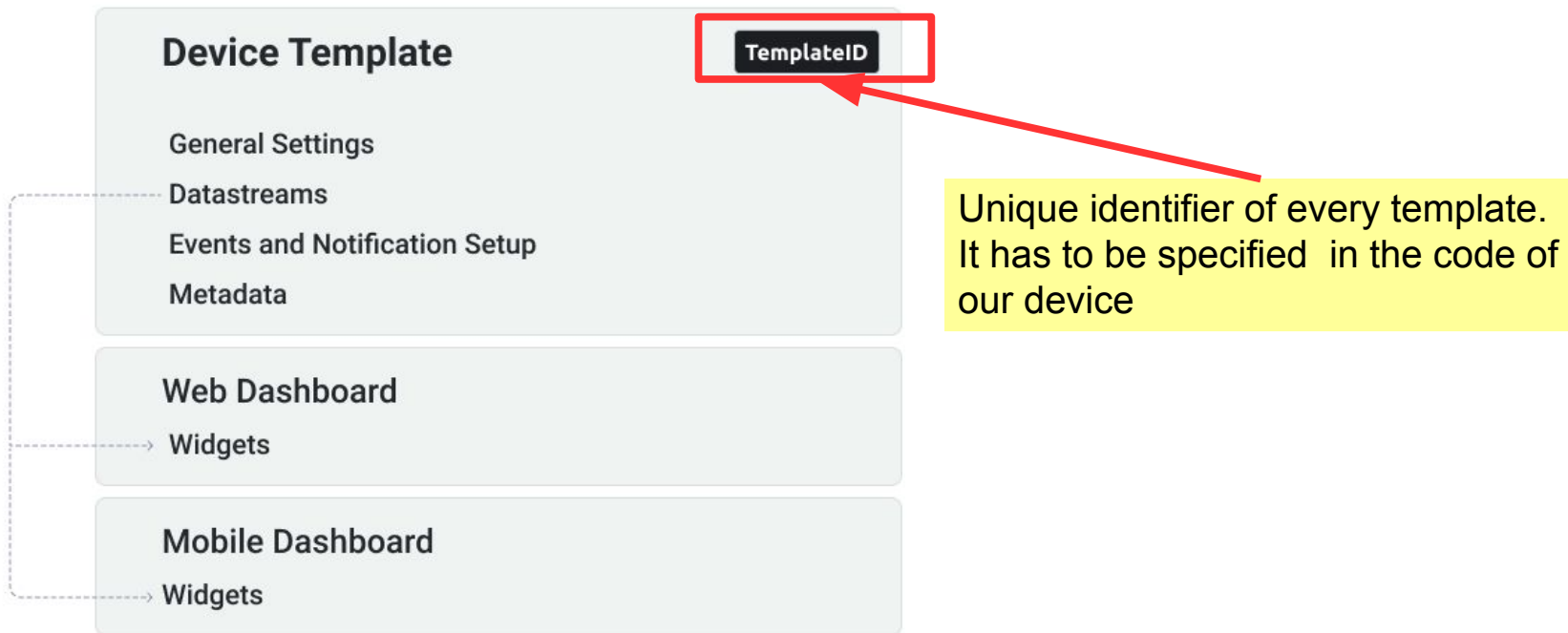
Device Template Concept

Unified management for multiple devices performing identical functions

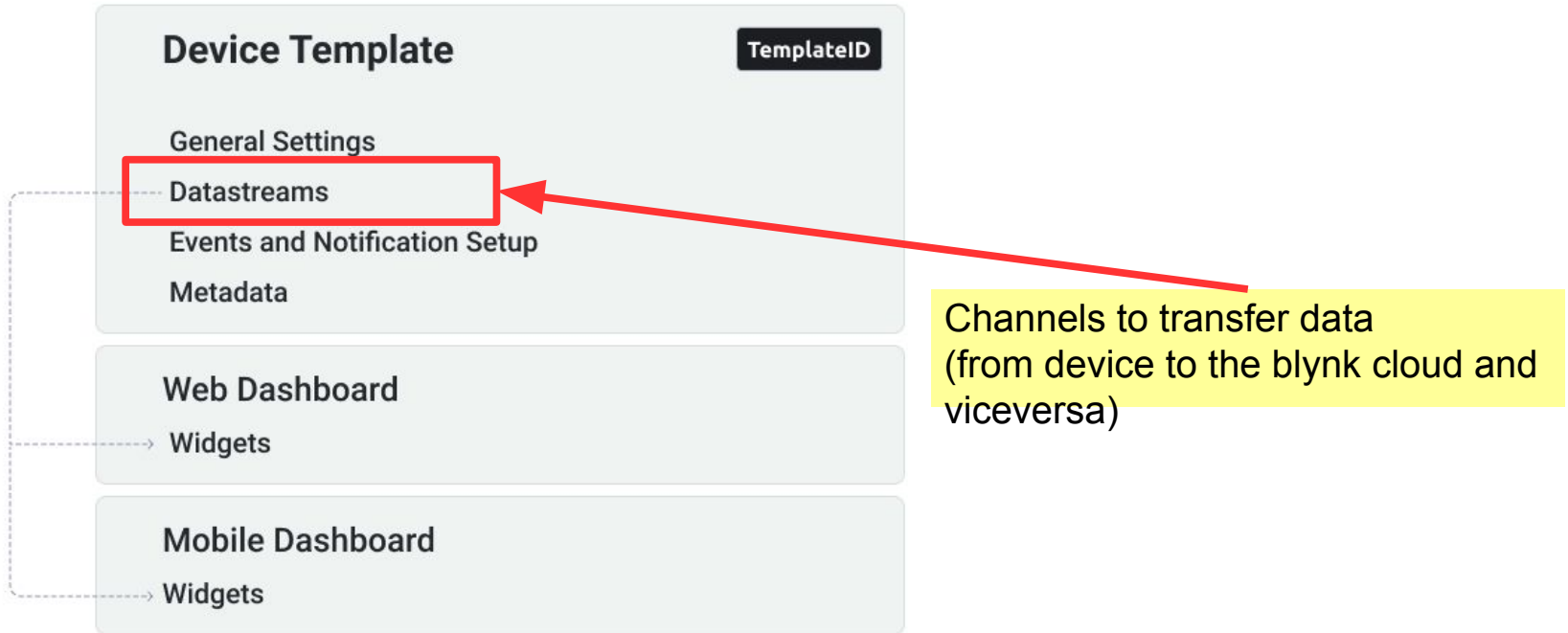


Allows developers to define a standard configuration once and apply it across an entire fleet of hardware.

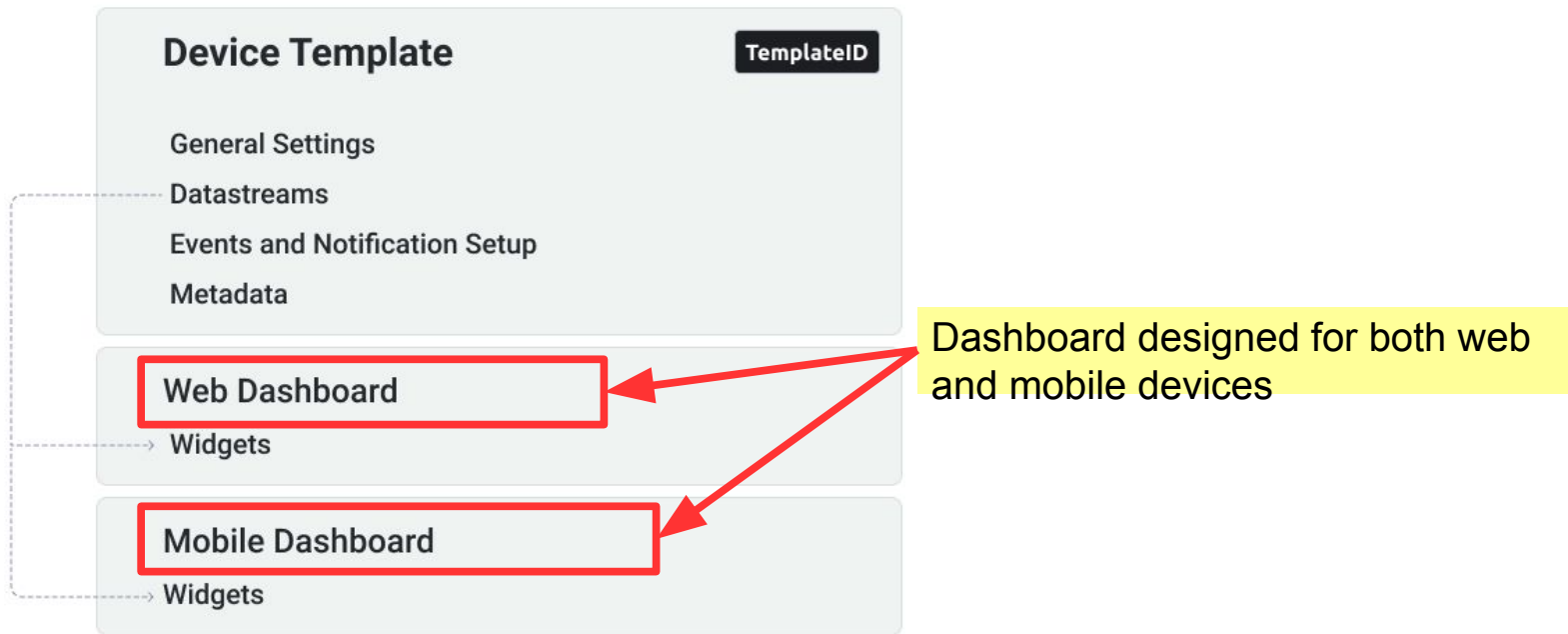
Device Template



Device Template



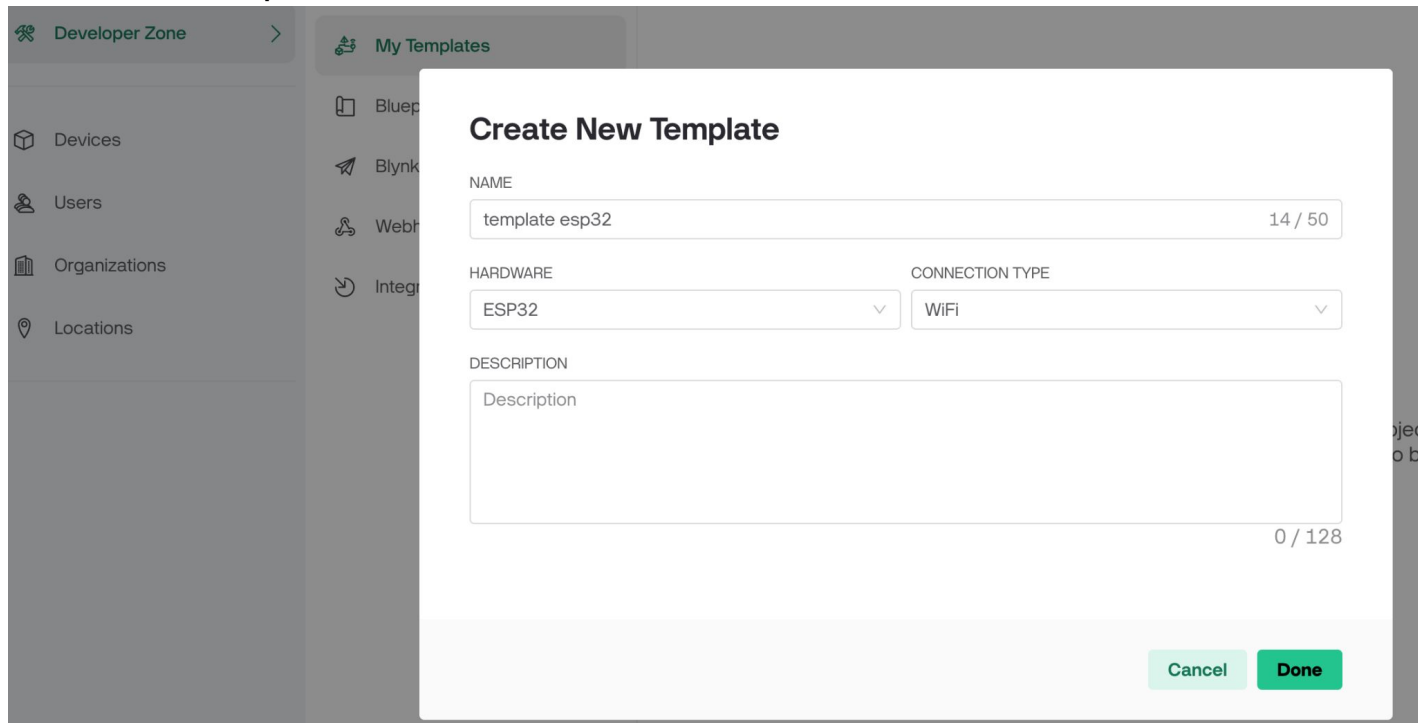
Device Template



Blynk: Create the Template

From Developer Zone:

- Create template for ESP32



The screenshot shows the Blynk Developer Zone interface with a 'Create New Template' modal open. The modal has a title 'Create New Template' and three input fields: 'NAME' (containing 'template esp32'), 'HARDWARE' (set to 'ESP32'), and 'CONNECTION TYPE' (set to 'WiFi'). Below these is a 'DESCRIPTION' text area. The background shows a sidebar with navigation links: 'Developer Zone', 'My Templates', 'Blueprints', 'Blynk', 'Webhooks', 'Integrations', 'Devices', 'Users', 'Organizations', and 'Locations'.

Create New Template

NAME
template esp32 14 / 50

HARDWARE CONNECTION TYPE
ESP32 WiFi

DESCRIPTION
Description 0 / 128

Cancel Done

Blynk: Create the Template

Always click “Save” !

×

☰

TEMPLATE ESP32

🏠 Home

📡 Datastreams

🖥️ Web Dashboard

☰ Metadata

📶 Connection Lifecycle

🔔 Events & Notifications

📖 User Guides

📱 Mobile Dashboard

📁 Assets

 template esp32

⋮

Cancel

Save

Home

What's next?

- ☐ [Configure template](#)
- ☐ [Set Up Datastreams](#)
- ☐ [Set up the Web Dashboard](#)
- ☐ [Add first Device](#)

Template settings

ESP32, WiFi 

Firmware configuration

 Template ID and Template Name should be declared at the very top of the firmware code.

```
#define BLYNK_TEMPLATE_ID
"TMPL4o7tsE6xo"
#define BLYNK_TEMPLATE_NAME "template
esp32"
```

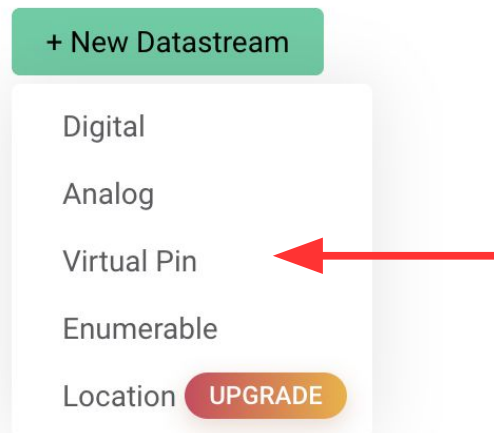
Example 1

- Switch on/off a LED in the ESP32 using your PC and/or your smartphone
 - Your Arduino, PC, smart phone are separated and located anywhere in the world!
 - Of course they have an active internet connection

Setup Datastream

Datastreams

Datastreams is a way to structure data that regularly flows in and out from device.
Use it for sensor data, any telemetry, or actuators.



Virtual Pins & Interface

- Just a switch
- The status of the switch is communicated from the UI to arduino by means of a Virtual Pin
 - Virtual Pin Datastream
 - V0, integer, can assume two values: 0 and 1

B



Setup Datastream

- You can choose different features and properties
- It depends on the project and the usage of that stream
- In this first example, we just want to communicate a boolean to drive remotely the LED status (on/off)

Virtual Pin Datastream

General

Expose to Automations



NAME

led pin

ALIAS

led pin

PIN

V0

DATA TYPE

Integer

UNITS

None

MIN

0

MAX

1

DEFAULT VALUE

0

☐

Enable history data

ADVANCED SETTINGS

Cancel

Create

Web Dashboard – Switch

Add the switch from the menu on the right, click on it for its settings and associate the data stream previously created (CLICK SAVE!)

×

☰

🏠

🔧

template esp32

Web Dashboard

Widget Box

CONTROL

Switch

🟢

Slider

— ○ — + 8

Number Input

— 0 +

Image Button

🖼️

Device Name Online

👤 Device Owner 🏢 Company Name

🕒 ⚙️ 📄 ⬇️ ⋮

Dashboard + 1t

Switch

led pin (V0)

Switch Settings

TITLE [OPTIONAL]

Switch LED

Datastream

Choose Datastream 🔍 or

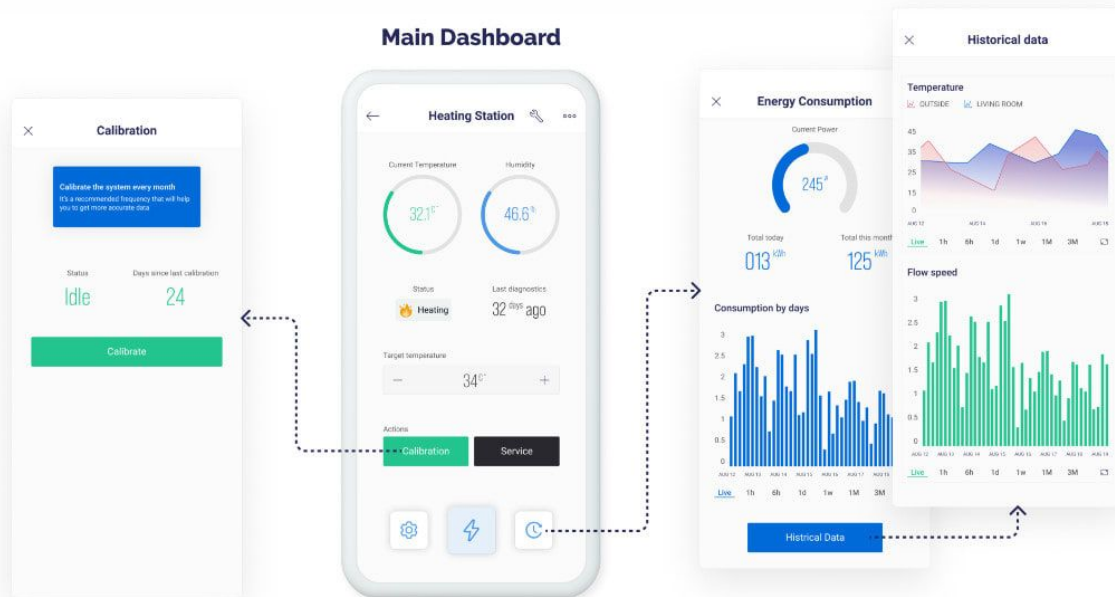
+ Create Datastream

led pin (V0)

led pin (V0)

Mobile Dashboard

You can also use Blynk IoT App



Add a Device

Instantiate a device from the template

My organization - 1246EU

DEVICES 1

My Devices 1

All 1

LOCATIONS 0

My Locations 0

All 0

USERS 1

My organization members 1

All 1

With no devices 0

My Devices

1 Device

+ New Device

Device name	De	Actions
Quickstart Device	Ma	

Region: fra1 [Privacy Policy](#)

Add a Device

New Device

Choose a way to create new device

From template



Scan QR code



Manual entry



New Device Created!

```
#define BLYNK_TEMPLATE_ID "TMPL4ocHl9Dl0"  
#define BLYNK_TEMPLATE_NAME "test2"  
#define BLYNK_AUTH_TOKEN "JVgf9t47ID7khtqnM-ewM3NBcj92RSol"
```

Template ID, Template Name, and AuthToken should be declared at the very top of the firmware code.

[Documentation](#)

[Copy to clipboard](#)

Testing with Code

Always consider the values provided by the blynk:

- Template ID
- Template Name
- Auth token



template esp32



Edit

Home

1 Devices

+ New Device

Device name	Status	Auth Token
device espino	● Offline	u64L - - -

Template settings

ESP32, WiFi



Firmware configuration



Template ID and Template Name should be declared at the very top of the firmware code.

```
#define BLYNK_TEMPLATE_ID "TMPL4o7tsE6xo"  
#define BLYNK_TEMPLATE_NAME "template esp32"
```

Testing with Code

- <https://examples.blynk.cc/>



Blynk

Board:
ESP32 ▼

Connection:
ESP32 WiFi ▼

Full list of supported hardware is [here](#)

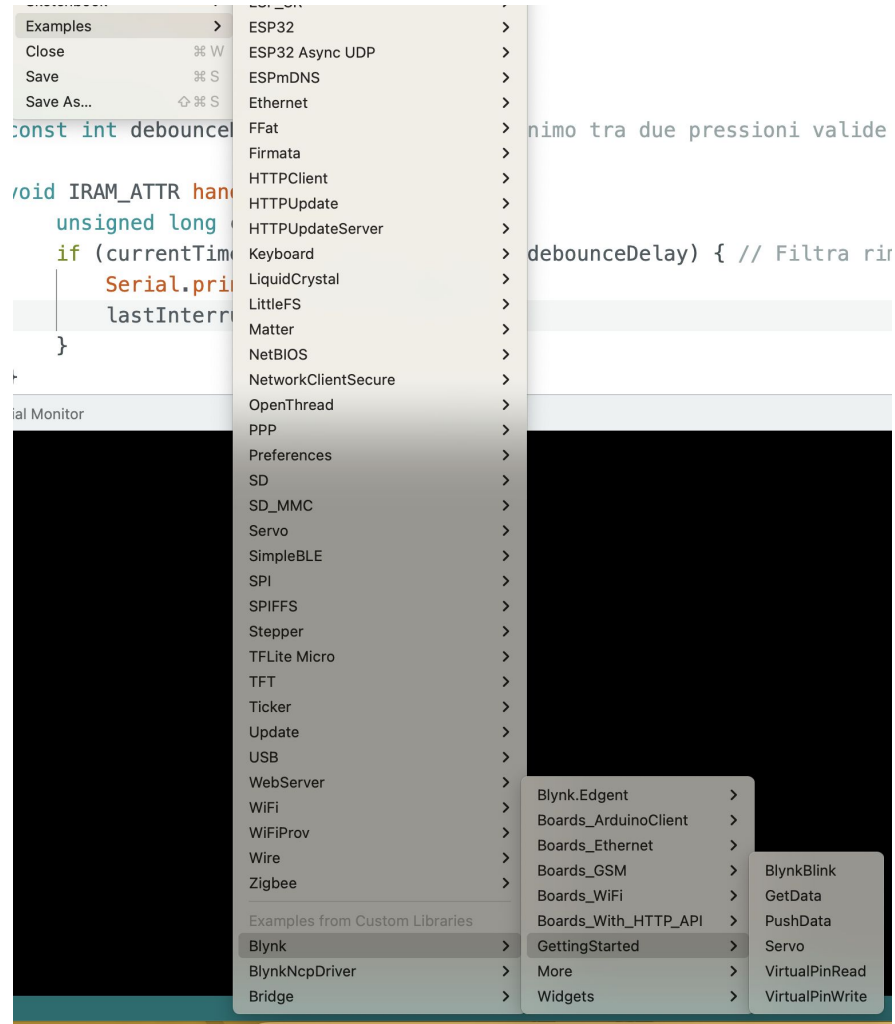
Template ID:

Template Name:

Auth Token:

Example:
Blynk Blink ▼

...Or load from IDE:



Testing with Code

Copy and paste Blynk parameters:

```
/* Fill in information from Blynk Device Info here */
```

```
#define BLYNK_TEMPLATE_ID      "TMPxxxxxx"  
#define BLYNK_TEMPLATE_NAME    "Device"  
#define BLYNK_AUTH_TOKEN      "YourAuthToken"
```

```
/* Comment this out to disable prints and save space */  
#define BLYNK_PRINT Serial
```

```
#include <WiFi.h>  
#include <WiFiClient.h>  
#include <BlynkSimpleEsp32.h>
```

```
// Your WiFi credentials.  
// Set password to "" for open networks.  
char ssid[] = "YourNetworkName";  
char pass[] = "YourPassword";
```

```
void setup()  
{  
  // Debug console  
  Serial.begin(115200);  
  
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);  
  // You can also specify server:  
  //Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass, "blynk.cloud", 80);  
  //Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass, IPAddress(192,168,1,100), 8080);  
}
```

```
#define BLYNK_TEMPLATE_ID "TMPL4B9417sf"  
#define BLYNK_DEVICE_NAME "remoted"  
#define BLYNK_AUTH_TOKEN  
"hZVwn1atoUuMMji9wDNGMoJP1bSi2pGz"
```

Testing with Code: Setup()

```
int pin_led = 23;

void setup()
{
  Serial.begin(115200);
  pinMode(pin_led, OUTPUT);
  Blynk.begin(BLYNK_AUTH_TOKEN, "Wokwi-GUEST", "");
}
```

Testing with Code: main loop

```
void loop() {  
  Blynk.run(); // Must be called as often as possible  
}
```

- Keeps the **connection alive** to the Blynk Cloud server
- Handles **incoming messages**, such as virtual pin writes (e.g., web dashboard)
- Manages **timed functions**, virtual pin reads, and syncs.
- You can put other things in the loop(), but avoid blocking code (e.g., long delay(), slow computations, etc)
- See the BlynkTimer topic on the next slides to deal with periodic actions without delays

BLYNK_WRITE: Virtual Pin Updates

```
// This function is called every time the Virtual Pin 0 state changes
BLYNK_WRITE(V0)
{
  // Set incoming value from pin V0 to a variable
  int value = param.asInt();

  if (value == 1)
    digitalWrite(13, HIGH);
  else
    digitalWrite(13, LOW);
}
```

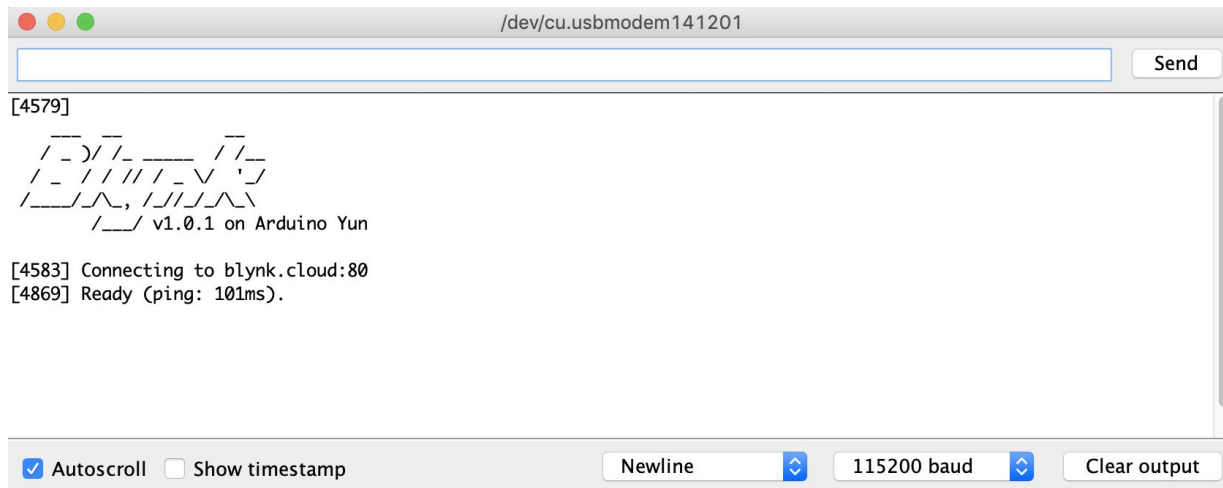
- “param”, is the name of the value passed to the virtual pin from the dashboard
- The code above depends on the type of value you set for the virtual pin
- See: <https://docs.blynk.io/en/blynk-library-firmware-api/virtual-pins>

Arduino Sketch

```
BLYNK_CONNECTED()  
{  
    Blynk.syncVirtual(V0); // will cause BLYNK_WRITE(V0)  
                           // to be executed  
}
```

Upload and Check

Upload the sketch and check in the terminal if blynk connection succeeds



```

#define BLYNK_TEMPLATE_ID          "TMPL4TCKLyvDX"
#define BLYNK_TEMPLATE_NAME        "espino"
#define BLYNK_AUTH_TOKEN           "ZL_3eEwqgYbmIEA4jDbhHg92A9uT0W94"

/* Comment this out to disable prints and save space */
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

char ssid[] = "Wokwi-GUEST";
char pass[] = "";

void setup()
{
  // Debug console
  Serial.begin(115200);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  pinMode(23, OUTPUT);
}

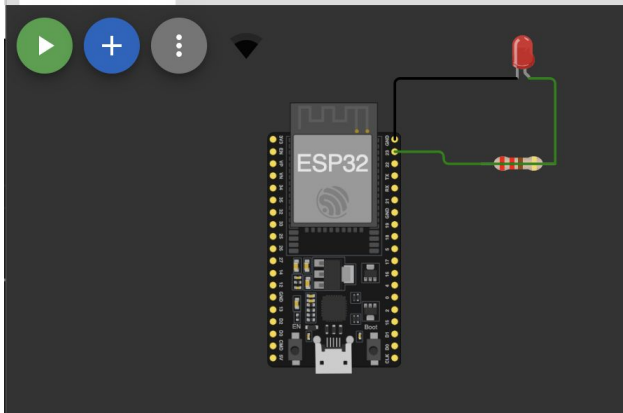
BLYNK_WRITE(V0) {
  int value = param.asInt();
  if (value==1)
    digitalWrite(23, HIGH);
  else
    digitalWrite(23, LOW);
}

BLYNK_CONNECTED() {
  Blynk.syncVirtual(V0);
}

void loop()
{
  Blynk.run();
}

```

Simulation



[1509] Connecting to Wokwi-GUEST

[6724] Connected to WiFi

[6724] IP: 10.10.0.2

[6725]

```

  _ _ _ _ _
 / _ ) / _ _ _ _ / / _
 / _ / / / / _ \ ' _/
 /___/_/_\_, /_/_/_/_\
          /___/ v1.3.2 on ESP32

```

#StandWithUkraine <https://bit.ly/swua>

[6736] Connecting to blynk.cloud:80

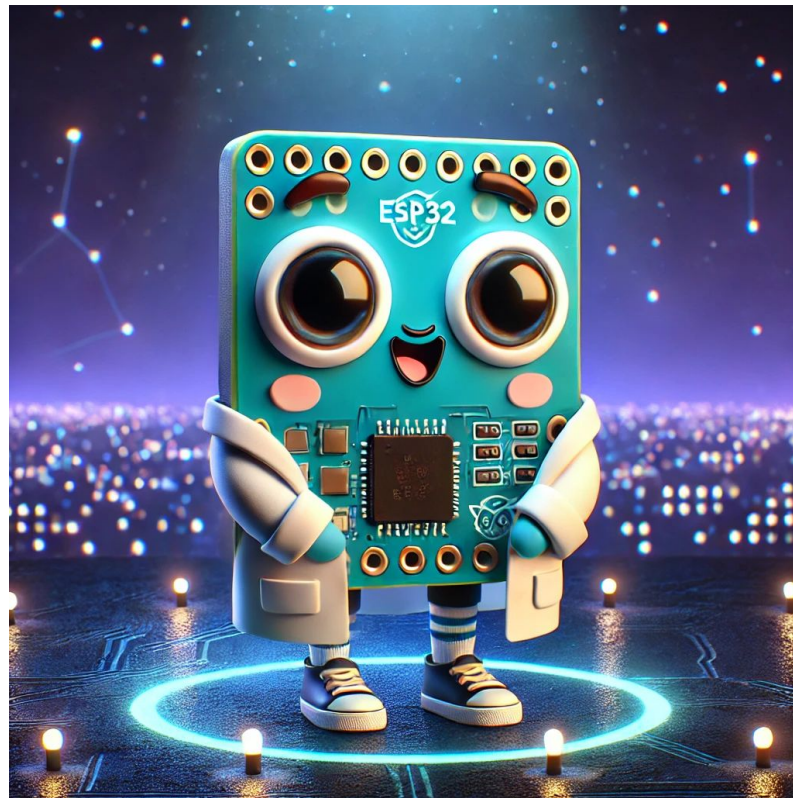
[7134] Redirecting to fra1.blynk.cloud:80

[7137] Connecting to fra1.blynk.cloud:80

[7368] Ready (ping: 48ms).

In Action...

Button



In Action...

Button



Writing Virtual Pin

- Device may send data to the Widgets over the Virtual Pin “pin”

```
1 Blynk.virtualWrite(pin, "abc");  
2 Blynk.virtualWrite(pin, 123); Blynk.virtualWrite(pin, 12.34);  
3 Blynk.virtualWrite(pin, "hello", 123, 12.34);
```

Exercise

*Add another data stream associated to the virtual pin V1
When the switch is activated, increase a counter and print its
value on the remote widget*



Exercise 2

```
BLYNK_WRITE(V0) {  
  int value = param.asInt();  
  if (value==1)  
    digitalWrite(23, HIGH);  
  else  
    digitalWrite(23, LOW);  
  
  Serial.println("Increasing TEST");  
  test++;  
  Blynk.virtualWrite(V1, test);  
}
```

Timers



```
#include <BlynkSimpleEsp32.h>
```

```
#define BLYNK_PRINT Serial  
char auth[] = "TUO_AUTH_TOKEN";  
char ssid[] = "TUO_WIFI";  
char pass[] = "TUA_PASSWORD";
```

```
BlynkTimer timer; // Creazione del timer
```

```
void myFunction() {  
  Serial.println("Timer Triggered!");  
  Blynk.virtualWrite(V1, millis() / 1000); // Aggiorna un widget su Blynk  
}
```

```
void setup() {  
  Serial.begin(115200);  
  Blynk.begin(auth, ssid, pass);  
  
  timer.setInterval(1000L, myFunction); // Esegue `myFunction` ogni 1000 ms  
}
```

```
void loop() {  
  Blynk.run(); // Mantiene la connessione con il server Blynk  
  timer.run(); // Gestisce i timer impostati  
}
```

```
void loop() {  
  Blynk.run();  
  Serial.println("Messaggio ogni 5 secondi");  
  delay(5000); // Blocca l'esecuzione e può causare problemi di connessione  
}
```

Just the Beginning...

Simulator: <https://wokwi.com/esp32>

Tutorials: <https://esp32io.com/>

Check the components available at unict:

<https://bit.ly/labunict>

